MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE, ALWAR

NAME OF FACULTY: RAJESH KUMAR

SUB & CODE: Database Management Systems (4CS4-04, 4AD4-04, 4AM4-04)

SEMESTER: IV SESSION: 2023-24 (Even SEM.)

BRANCH: CS & AI/DS BATCH: A & B

UNIT: II SUBMISSON: 23-2-2024

Lecture No.	Topics Covered	Page No.
1	Relationship Algebra Selection and Projection	1-2
2	Set Operations, Renaming, Joints,Divisions	3-4
3	Relation Calculus, Expressive Power of Algebra and Calculus.	5-9
4	SQL queries programming and Triggers:The Forms of a Basic SQL Query, Union, and Intersection and Except	10-14
5	Nested Queries, Correlated Nested Queries	15-17
6	Set-Comparison Operations and Aggregate Operators	18-21
7	Null Values and Embedded SQL, Dynamic SQL	22-25
8	ODBC and JDBC	26-29
9	Triggers and Active Databases	30-32

References:

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F Korth
2. Fundamentals of Database Systems by Elmsari, Navathe, 5th Edition Pearson

Faculty HOD Dean Academics 16/3/24

UNIT-2 (Relation Algebra & Selection)
LECTURE NO: -12

RELATIONAL ALGEBRA is a widely used procedural query language. It collects instances of relations as input and gives occurrences of relations as output. It uses various operations to perform this action. SQL Relational algebra query operations are performed recursively on a relation. The output of these operations is a new relation, which might be formed from one or more input relations.

Basic SQL Relational Algebra Operations

Relational Algebra divided in various groups

Unary Relational Operations

SELECT (symbol: σ)

PROJECT (symbol: π)

RENAME (symbol: ρ)

Relational Algebra Operations from Set Theory

UNION (υ)

INTERSECTION ( )

DIFFERENCE (-)

CARTESIAN PRODUCT (x)

Binary Relational Operations

JOIN

DIVISION

1. Select Operation:

The select operation selects tuples that satisfy a given predicate.

It is denoted by sigma (σ).

Notation: σ p(r)

Where:

σ is used for selection prediction

r is used for relation

p is used as a propositional logic formula which may use connectors like: AND OR and NOT. These relational can use as relational operators like =, ≠, ≥, <, >, ≤.

MITRC

For example: LOAN Relation

BRANCH_NAME	LOAN_NO	AMOUNT
Downtown	L-17	1000
Redwood	L-23	2000
Perryride	L-15	1500
Downtown	L-14	1500
Perryride	L-16	1300

σ BRANCH_NAME="perryride" (LOAN)

Output:

BRANCH_NAME	LOAN_NO	AMOUNT
Perryride	L-15	1500
Perryride	L-16	1300

2. Project Operation:

This operation shows the list of those attributes that we wish to appear in the result. Rest of the attributes are eliminated from the table.

It is denoted by ∏.

Notation: ∏ A1, A2, An (r)

Where A1, A2, A3 is used as an attribute name of relation r.

Example: CUSTOMER RELATION

NAME	STREET	CITY
Jones	Main	Harrison
Smith	North	Rye
Hays	Main	Harrison
Curry	North	Rye

MITRC

Input:

∏ NAME, CITY (CUSTOMER)

Input:

∏ NAME, CITY (CUSTOMER)

Output:

NAME	CITY
Jones	Harrison
Smith	Rye
Hays	Harrison
Curry	Rye

MITRC

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth

2.Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

3. C.J. Date: Data Base Design, Addison Wesley

UNIT-2 (Set Operations, Renaming, and Joints, Divisions)

LECTURE NO: -13

SET Operations in SQL

SQL supports few Set operations which can be performed on the table data. These are used to get meaningful results from data stored in the table, under different special conditions.

Different types of SET operations, along with example:

1. UNION
2. UNION ALL
3. INTERSECT
4. MINUS

1. Union: - Union combines two different results obtained by a query into a single result in the form of a table. However, the results should be similar if union is to be applied on them. Union removes all duplicates, if any from the data and only displays distinct values.

The First table

ID	Name
1	abhi
2	adam

The Second table

ID	Name
2	adam
3	Chester

Union SQL query will be:-

```
SELECT * FROM First  UNION SELECT * FROM Second;
```

The result set table will look like

ID	NAME
1	abhi
2	adam
3	Chester

2.Intersection

The intersection operator gives the common data values between the two data sets that are intersected. The two data sets that are intersected should be similar for the intersection operator to work. Intersection also removes all duplicates before displaying the result.

Intersect query will be :-

SELECT * FROM First INTERSECT SELECT * FROM Second;

The result set table will look like

ID	NAME
2	adam

3.UNION ALL: -This operation is similar to Union. But it also shows the duplicate rows.

SELECT * FROM First UNION ALL SELECT * FROM Second;

The resultset table will look like,

The resultset table will look like,

ID	NAME
1	abhi
2	adam
2	adam
3	Chester

4. MINUS: -The Minus operation combines results of two SELECT statements and return only those in the final result, which belongs to the first set of the result.

SELECT * FROM First MINUS SELECT * FROM Second;

The resultset table will look like,

ID	NAME
1	abhi

Rename Operation:

The rename operation is used to rename the output relation. It is denoted by rho (ρ).

Student: -

Stud_Id	Stud_Name
12	Shyam
11	Ram

Example: We can use the rename operator to rename STUDENT relation to STUDENT1.

ρ(STUDENT1, STUDENT)

Division Operator (÷): Division operator A÷B can be applied if and only if:

Attributes of B is proper subset of Attributes of A.

The relation returned by division operator will have attributes = (All attributes of A – All Attributes of B)

The relation returned by division operator will return those tuples from relation A which are associated to every B's tuple.

Consider the relation STUDENT_SPORTS and ALL_SPORTS

STUDENT_SPORTS

ROLL_NO	SPORTS
1	Badminton
2	Cricket
2	Badminton
4	Badminton

MITRC

ALL_SPORTS

SPORTS
Badminton
Cricket

To apply division operator as

STUDENT_SPORTS÷ ALL_SPORTS

The operation is valid as attributes in ALL_SPORTS is a proper subset of attributes in STUDENT_SPORTS.

The attributes in resulting relation will have attributes {ROLL_NO, SPORTS}-{SPORTS}=ROLL_NO

The tuples in resulting relation will have those ROLL_NO which are associated with all B's tuple {Badminton, Cricket}. ROLL_NO 1 and 4 are associated to Badminton only. ROLL_NO 2 is associated to all tuples of B. So the resulting relation will be:

ROLL_NO
2

For example:- R1

Name	**Course**
System	Btech
Database	Mtech
Database	Btech
Algebra	Btech

÷

R2

Course
Btech
Btech

MITRC

 Set Operations, Renaming, and Joints, Divisions

=

Name
database

Join: - Join in DBMS is a binary operation which allows you to combine join product and selection in one single statement. The goal of creating a join condition is that it helps you to combine the data from two or more DBMS tables. The tables in DBMS are associated using the primary key and foreign keys.

Types of Join

- Inner Joins: Theta, Natural
- Outer Join: Left, Right, Full

1.Inner Join:-

INNER JOIN is used to return rows from both tables which satisfy the given condition. It is the most widely used join operation and can be considered as a default join-type.

An Inner join or equijoin is a comparator-based join which uses equality comparisons in the join-predicate. Inner Join further divided into two subtypes:

Theta join

Natural join

2.Outer Joins: -

Theta Join, Equijoin, and Natural Join are called inner joins. An inner join includes only those tuples with matching attributes and the rest are discarded in the resulting relation.

Therefore, we need to use outer joins to include all the tuples from the participating relations in the resulting relation. There are three kinds of outer joins.

1. left outer join
2. right outer join
3. full outer join.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth

2.Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

MITRC

UNIT-2 (Relation Calculus, Expressive Power of Algebra and Calculus)

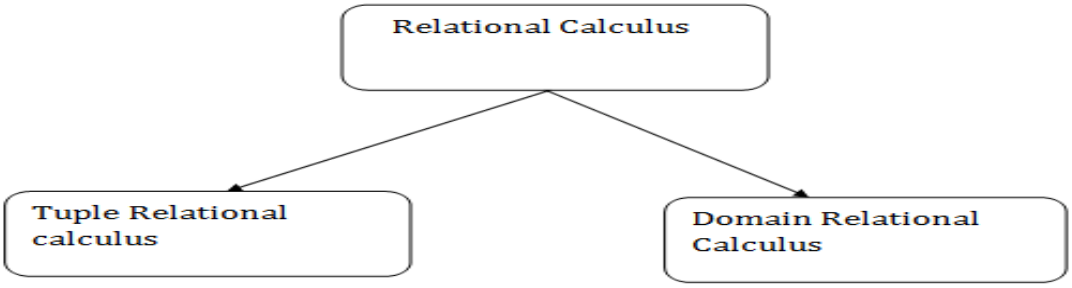
LECTURE NO: -14

Relational Calculus

Relational calculus is a non-procedural query language, and instead of algebra, it uses mathematical predicate calculus. The relational calculus is not the same as that of differential and integral calculus in mathematics but takes its name from a branch of symbolic logic termed as predicate calculus. Relational calculus is a non-procedural query language. In the non-procedural query language, the user is concerned with the details of how to obtain the end results.

The relational calculus tells what to do but never explains how to do.

Types of Relational calculus:



1. Tuple Relational Calculus (TRC)

The tuple relational calculus is specified to select the tuples in a relation. In TRC, filtering variable uses the tuples of a relation.

The result of the relation can have one or more tuples.

Notation:

{T | P (T)} or {T | Condition (T)}

Where

T is the resulting tuples

P(T) is the condition used to fetch T.

For example: -

{T.name | Author(T) AND T. article = 'database' }

OUTPUT: This query selects the tuples from the AUTHOR relation. It returns a tuple with 'name' from Author who has written an article on 'database'.

 Relation Calculus, Expressive Power of Algebra and Calculus.

2. Domain Relational Calculus (DRC)

The second form of relation is known as Domain relational calculus. In domain relational calculus, filtering variable uses the domain of attributes.

Domain relational calculus uses the same operators as tuple calculus. It uses logical connectives ∧ (and), ∨ (or) and ¬ (not).

It uses Existential (∃) and Universal Quantifiers (∀) to bind the variable.

For example:

{< article, page, subject > | ∈ DBMS ∧ subject = 'database'}

Output: This query will yield the article, page, and subject from the relational DBMS, where the subject is a database.

Difference between Relational Algebra and Relational Calculus:

S.NO	Relational Algebra	Relational Calculus
1.	It is a Procedural language.	While Relational Calculus is Declarative language.
2.	Relational Algebra means how to obtain the result.	While Relational Calculus means what result we have to obtain.
3.	In Relational Algebra, The order is specified in which the operations have to be performed.	While in Relational Calculus, The order is not specified.
4.	Relational Algebra is independent of the domain.	While Relation Calculus can be domain-dependent.
5.	Relational Algebra is nearer to a programming language.	While Relational Calculus is not nearer to programming language.

MITRC

 Relation Calculus, Expressive Power of Algebra and Calculus.

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth

2.Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

MITRC

UNIT-2 (SQL Queries Programming and Triggers)
LECTURE NO: -15

SQL: - SQL is a database computer language designed for the retrieval and management of data in a relational database. SQL stands for Structured Query Language.

Why to Learn SQL?

SQL is Structured Query Language, which is a computer language for storing, manipulating and retrieving data stored in a relational database.

SQL is the standard language for Relational Database System. All the Relational Database Management Systems (RDMS) like MySQL, MS Access, Oracle, Sybase, Informix, Postgres and SQL Server use SQL as their standard database language.

Applications of SQL

As mentioned before, SQL is one of the most widely used query language over the databases. I'm going to list few of them here:

- Allows users to access data in the relational database management systems.
- Allows users to describe the data.
- Allows users to define the data in a database and manipulate that data.
- Allows to embed within other languages using SQL modules, libraries & pre-compilers.
- Allows users to create and drop databases and tables.

A Brief History of SQL

- 1970 – Dr. Edgar F. "Ted" Codd of IBM is known as the father of relational databases. He described a relational model for databases.
- 1974 – Structured Query Language appeared.
- 1978 – IBM worked to develop Codd's ideas and released a product named System/R.
- 1986 – IBM developed the first prototype of relational database and standardized by ANSI. The first relational database was released by Relational Software which later came to be known as Oracle.

MITRC

Union, and Intersection: -

Union

Union combines two different results obtained by a query into a single result in the form of a table. However, the results should be similar if union is to be applied on them. Union removes all duplicates, if any from the data and only displays distinct values.

The First table

ID	Name
1	abhi
2	adam

The Second table

ID	Name
2	adam
3	Chester

Union SQL query will be:-

SELECT * FROM First UNION SELECT * FROM Second;

The result set table will look like

ID	NAME
1	abhi
2	adam
3	Chester

Intersection

The intersection operator gives the common data values between the two data sets that are intersected. The two data sets that are intersected should be similar for the intersection operator to work. Intersection also removes all duplicates before displaying the result.

Triggers

Triggers are the SQL statements that are automatically executed when there is any change in the database. The triggers are executed in response to certain events (INSERT, UPDATE or DELETE) in a particular table. These triggers help in maintaining the integrity of the data by changing the data of the database in a systematic fashion.

Advantages of Triggers

- Triggers provide a way to check the integrity of the data. When there is a change in the database the triggers can adjust the entire database.
- Triggers help in cheeking User Interface lightweight. Instead of putting the same function call all over the application you can put a trigger and it will be executed.

Disadvantages of Triggers

- Triggers may be difficult to troubleshoot as they execute automatically in the database. If there is some error then it is hard to find the logic of trigger because they are fired before or after updates/inserts happen.
- The triggers may increase the overhead of the database as they are executed every time any field is updated.

MITRC

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth

2.Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

UNIT-2 (Nested Queries, Correlated Nested Queries)
LECTURE NO: -16

Nested Query –

In Nested Query, Inner query runs first, and only once. Outer query is executed with result from Inner query. Hence, Inner query is used in execution of Outer query. We will use STUDENT, COURSE, STUDENT_COURSE tables for understanding nested queries.

STUDENT

S_ID	S_NAME	S_ADDRESS	S_PHONE	S_AGE
S1	RAM	DELHI	9455123451	18
S2	RAMESH	GURGAON	9652431543	18
S3	SUJIT	ROHTAK	9156253131	20
S4	SURESH	DELHI	9156768971	18

COURSE

C_ID	C_NAME
C1	DSA
C2	Programming
C3	DBMS

STUDENT_COURSE

S_ID	C_ID
S1	C1
S1	C3

S2 C1

S3 C2

S4 C2

S4 C3

There are mainly two types of nested queries:

Independent Nested Queries: In independent nested queries, query execution starts from innermost query to outermost queries. The execution of inner query is independent of outer query, but the result of inner query is used in execution of outer query. Various operators like IN, NOT IN, ANY, ALL etc are used in writing independent nested queries.

Orders (OrderID, CustomerID, OrderDate);

Customers (CustomerID, CustomerName, ContactName, Country);

Find details of customers who have ordered.

SELECT * FROM Customers WHERE

CustomerID IN (SELECT CustomerID FROM Orders);

Correlated Query: -

In Correlated Query, Outer query executes first and for every Outer query row Inner query is executed. Hence, Inner query uses values from Outer query.

Example –

Orders (OrderID, CustomerID, OrderDate);

Customers (CustomerID, CustomerName, ContactName, Country);

Find details of customers who have ordered.

SELECT * FROM Customers where

EXISTS (SELECT CustomerID FROM Orders

WHERE Orders.CustomerID=Customers.CustomerID);

MITRC

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth

2.Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education (2008).

MITRC

UNIT-2 (Set-Comparison Operations and Aggregate Operators)
LECTURE NO: -17

SQL supports few Set operations which can be performed on the table data. These are used to get meaningful results from data stored in the table, under different special conditions.

In this tutorial, we will cover 4 different types of SET operations, along with example:

1. UNION
2. UNION ALL
3. INTERSECT

UNION Operation

UNION is used to combine the results of two or more SELECT statements. However, it will eliminate duplicate rows from its result set. In case of union, number of columns and data type must be same in both the tables, on which UNION operation is being applied.

Example of UNION

The First table

ID	Name
1	abhi
2	adam

The Second table

ID	Name
2	adam
3	Chester

Union SQL query will be: -

```
SELECT * FROM First UNION SELECT * FROM Second;
```

The result set table will look like

NITRC

ID	NAME
1	abhi
2	adam
3	Chester

2. UNION ALL

This operation is similar to Union. But it also shows the duplicate rows.

The First table

ID	Name
1	abhi
2	adam

The Second table

ID	Name
2	adam
3	Chester

Union All query will be like

SELECT * FROM First UNION ALL SELECT * FROM Second;

The result set table will look like,

ID	NAME
1	abhi
2	adam
2	adam
3	Chester

MITRC

 Set-Comparison Operations and Aggregate Operators

3. INTERSECT

Intersect operation is used to combine two SELECT statements, but it only retunes the records which are common from both SELECT statements. In case of Intersect the number of columns and data type must be same.

NOTE: MySQL does not support INTERSECT operator.

The First table

ID	Name
1	abhi
2	adam

The Second table

ID	Name
2	adam
3	Chester

Intersect query will be:-

SELECT * FROM First INTERSECT SELECT * FROM Second;

The result set table will look like

ID	NAME
2	adam

4. Minus

It combines the result of two SELECT statements. Minus operator is used to display the rows which are present in the first query but absent in the second query.

It has no duplicates and data arranged in ascending order by default.

Syntax:

SELECT column_name FROM table1

MINUS

SELECT column_name FROM table2;

Example: Using the above First and Second table.

SELECT * FROM First

MINUS

SELECT * FROM Second;

The result set table will look like:

ID	NAME
1	Jack
2	Harry

SQL Aggregate Functions

SQL aggregation function is used to perform the calculations on multiple rows of a single column of a table. It returns a single value.

It is also used to summarize the data.

1. COUNT FUNCTION

COUNT function is used to Count the number of rows in a database table. It can work on both numeric and non-numeric data types.

COUNT function uses the COUNT(*) that returns the count of all the rows in a specified table. COUNT(*) considers duplicate and Null.

PRODUCT_MAST Table: -

PRODUCT	COMPANY	QTY	RATE	COST
Item1	Com1	2	10	20
Item2	Com2	3	25	75
Item3	Com1	2	30	60
Item4	Com3	5	10	50
Item5	Com2	2	20	40

MITRC

SELECT COUNT (*) FROM PRODUCT_MAST;

Output: - 05

2. **SUM Function: -** Suppose we want a report that gives total amount of payments made so far. We can use the SQL SUM function which returns the sum of all the values in the specified column. SUM works on numeric fields only. Null values are excluded from the result returned.

SELECT SUM(COST) FROM PRODUCT_MAST;

Output: - 245

3. **AVG function: -**SQL AVG function returns the average of the values in a specified column. Just like the SUM function, it works only on numeric data types. AVG function returns the average of all non-Null values.

SELECT AVG(COST) FROM PRODUCT_MAST;

Output: - 49.00

4. **MAX Function: -** Just as the name suggests, the MAX function is the opposite of the MIN function. It returns the largest value from the specified table field. Let's assume we want to get the year that the latest movie in our database was released. We can easily use the MAX function to achieve that.

SELECT MAX(RATE) FROM PRODUCT_MAST;

Output: - 30

5. **MIN Function: -** The MIN function returns the smallest value in the specified table field. As an example, let's suppose we want to know the year in which the oldest movie in our library was released, we can use SQL's MIN function to get the desired information.

SELECT MIN(RATE) FROM PRODUCT_MAST;

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.

3. C.J. Date: Data Base Design, Addison Wesley.

MITRC

UNIT-2 (Null Values and Embedded & Dynamic SQL)

LECTURE NO: -18

In SQL there may be some records in a table that do not have values or data for every field. This could be possible because at a time of data entry information is not available. So SQL supports a special value known as NULL which is used to represent the values of attributes that may be unknown or not apply to a tuple. SQL places a NULL value in the field in the absence of a user-defined value.

For example, the Apartment_number attribute of an address applies only to address that are in apartment buildings and not to other types of residences.

Importance of NULL value

- It is important to understand that a NULL value is different from a zero value.
- A NULL value is used to represent a missing value, but that it usually has one of three different interpretations:
- The value unknown (value exists but is not known)
- Value not available (exists but is purposely withheld)
- Attribute not applicable (undefined for this tuple)
- It is often not possible to determine which of the meanings is intended. Hence, SQL does not distinguish between the different meanings of NULL.

Where SQL is NULL

There is an example of student table:

SURNAME	NAME	MARKS
TYAGI	SEEMA	
SINGH	RAMAN	5
SHARMA	AMAR	
JAISWAL	VICKY	6

SELECT SURNAME, NAME, MARKS FROM STUDENTS

WHERE MARKS IS NULL;

It will return the following records:

SURNAME	NAME	MARKS
SHARMA	AMAR	
TYAGI	SEEMA	

Where SQL is NOT NULL

How to select records with no null values (in marks column)? Let's see the query to get all the records where marks is NOT NULL

SELECT SIR_NAME, FIRSTNAME, MARKS FROM STUDENTS

WHERE MARKS IS NOT NULL ;

SURNAME	NAME	MARKS
SINGH	RAMAN	5
JAISWAL	VICKY	6

Embedded SQL

Database languages are meant for dealing with databases. They are used only to query tables / views, manipulate the data values using insert/update/ delete, or set of transactions can be executed one after the other to modify set of tables/view. But all these transactions can be performed by a good database developer or a SQL programmer who has good knowledge about database as well as SQL.

But in real time, these developer or programmers will not be using the database. Actual use of DB is for normal user for whom DB is a black box. In order to make easy for the normal user, applications, UI or forms are created where user can enter his values or requirement. The underlying application program will manipulate his request. Hence user need not have any knowledge about the DB.

But how application communicates with DB? Applications are developed using some general-purpose languages like C, C++, JAVA, etc. Similarly, DB languages like SQL, SQL servers etc are

meant for only DB activities. Hence both application language and DB language are completely different from each other. But in order to make application programs to work, DB is compulsory; and for any DB to be used by the user application/UI is a must. Hence both are dependent on each other too. This gap between application programs and SQL is bridged by the use of embedded SQL and dynamic SQLs.

These SQLs provide the utility to use SQLs inside the application language like C, C++, Java etc, and make these applications to communicate with DB. Hence when user submits a request or enters values in the form, he gets the result what he is requested.

Advantages of Embedded SQL

Some of the advantages of using SQL embedded in high-level languages are as follows:

- Helps to access databases from anywhere.
- Allows integrating authentication service for large scale applications.
- Provides extra security to database transactions.
- Avoids logical errors while performing transactions on our database.
- Makes it easy to integrate the frontend and the backend of our application.

What is Dynamic SQL?

Dynamic SQL is the process that we follow for programming SQL queries in such a way that the queries are built dynamically with the application operations.

It helps us to manage big industrial applications and manage the transactions without any added overhead.

With dynamic SQL we are free to create flexible SQL queries and the names of the variables or any other parameters are passed when the application runs.

Dynamic SQL is a programming technique that enables you to build SQL statements dynamically at runtime. You can create more general purpose, flexible applications by using dynamic SQL because the full text of a SQL statement may be unknown at compilation.

differences between Static or Embedded and Dynamic or Interactive SQL

Static (Embedded) SQL	Dynamic (Interactive) SQL
In Static SQL, how database will be accessed is predetermined in the embedded SQL statement.	In Dynamic SQL, how database will be accessed is determined at run time.
It is swifter and more efficient.	It is less swift and efficient.

MITRC

SQL statements are compiled at compile time.	SQL statements are compiled at run time.
Parsing, Validation, Optimization and Generation of application plan are done at compile time.	Parsing, Validation, Optimization and Generation of application plan are done at run time.
It is generally used for situations where data is distributed uniformly.	It is generally used for situations where data is distributed non uniformly.
EXECUTE IMMEDIATE, EXECUTE and PREPARE statements are not used.	EXECUTE IMMEDIATE, EXECUTE and PREPARE statements are used.

MITRC

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth

UNIT-2 (ODBC & JDBC)
LECTURE NO: -19

Definition of JDBC

Java Database Connectivity (JDBC) is an application programming interface i.e. (API). JDBC was released as a part of Java development Kit (JDK) 1.1. in the year 1996 by SUN Microsoft.

It is a standard interface between any Java application and different databases. The function of JDBC is to help the Java-based application to access different types of databases. JDBC provide methods to query database, and it can also be used to update the database. JDBC provide JDBC drivers that converts the request from Java application on client side to the language that database understands.

As JDBC is language and platform specific, Java application can use JDBC-to-ODBC bridge to communicate with ODBC adaptable databases.

Definition of ODBC

ODBC is Open Database Connectivity. Like JDBC, ODBC is also an API that acts as an interface between an application on the client side and the database on the server side. Microsoft introduced ODBC in the year 1992.

ODBC helps an application to access the data from the database. An application written in any language can use ODBC to access different types of databases and hence, it is said to be language and platform independent. Like JDBC, ODBC also provides ODBC drivers that convert the request of application written in any language into the language understandable by databases.

differences between ODBC and JDBC

Sr. No.	Key	ODBC	JDBC
1	Stands For	ODBC stands for Open Database Connectivity which literally means that it is compatible with all types of languages such as C, C++, Java, etc.	JDBC Stands for Java database connectivity i.e only compatible with java language.
2	Introduction	ODBC was introduced by Microsoft prior to JDBC in 1992.	JDBC was introduced by SUN Microsystems after ODBC in 1997.

Sr. No.	Key	ODBC	JDBC
3	Platform dependency	ODBC is platform dependent as we can use ODBC only for windows platform.	On the other hand, JDBC is platform-independent and can be used for any platform.
4	Type	ODBC can be considered as a type of procedural as most of these drivers are developed in a native language like C and C++ which are the procedural types of language.	On the other hand, JDBC is a purely object-oriented type driver.
5	Performance	The performance of ODBC is faster as compared to JDBC as data imports and exports are faster and memory-intensive.	On the other hand, performance of JDBC is slower than native ODBC but its platform independence allowing to work with any operating system (including Mac and Linux).

MITRC

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.
2. Fundamentals of Database Systems" by Elmsari, Navathe, 5th Edition, Pearson Education 2008).

UNIT-2 (Trigger and Active Databases)
LECTURE NO: -20

What is a trigger?

Trigger is like a procedure that is automatically invoked by the DBMS in response to specified changes to data base. Trigger is like a 'Daemon that monitors a data base, and is executed when the data base is modified in a way that matches the event specification. A data base that has a set of associated triggers is called an active data base.

Syntax

create trigger Trigger_name

(before | after)

[insert | update | delete]

on [table_name]

[for each row]

[trigger_body]

Parts of a Trigger

Whenever a trigger is created, it contains the following three sequential parts:

Triggering Event or Statement: - The statements due to which a trigger occurs is called triggering event or statement. Such statements can be DDL statements, DML statements or any database operation, executing which gives rise to a trigger.

Trigger Restriction: - The condition or any limitation applied on the trigger is called trigger restriction. Thus, if such a condition is TRUE then trigger occurs otherwise it does not occur.

Trigger Action: The body containing the executable statements that is to be executed when trigger occurs that is with the execution of Triggering statement and upon evaluation of Trigger restriction as **True** is called Trigger Action.

Types of Triggers

Depending upon, when a trigger is fired, it may be classified as :

1. Statement-level trigger
2. Row-level trigger
3. Before triggers
4. After triggers
5. Combination Trigger

1. Statement-level Trigger

MITRC

A statement trigger is fired only for once for a DML statement irrespective of the number of rows affected by the statement. For example, if you execute the following UPDATE command STUDENTS table, statement trigger for UPDATE is executed only for once.

update students set bcode='b3'

where bcode = 'b2';

However, statements triggers cannot be used to access the data that is being inserted, updated or deleted. In other words, they do not have access to keywords NEW and OLD, which are used to access data. Statement-level triggers are typically used to enforce rules that are not related to data. For example, it is possible to implement a rule that says "nobody can modify BATCHES table after 9 P.M". Statement-level trigger is the default type of trigger.

2. Row-level Trigger

A row trigger is fired once for each row that is affected by DML command. For example, if an UPDATE command updates 100 rows then row-level trigger is fired 100 times whereas a statement-level trigger is fired only for once.

Row-level trigger are used to check for the validity of the data. They are typically used to implement rules that cannot be implemented by integrity constraints. Row-level triggers are implemented by using the option FOR EACH ROW in CREATE TRIGGER statement

3. Before Triggers

While defining a trigger, you can specify whether the trigger is to be fired before the command (INSERT, DELETE, and UPDATE) is executed or after the command is executed. Before triggers are commonly used to check the validity of the data before the action is performed. For instance, you can use before trigger to prevent deletion of row if deletion should not be allowed in the given case.

4. AFTER Triggers

After triggers are fired after the triggering action is completed. For example, If after trigger is associated with INSERT command then it is fired after the row is inserted into the table.

5. Combination Trigger: Combination trigger are combination of two trigger type, Before Statement Trigger: Trigger fire only once for each statement before the triggering DML statement.

Before Row Trigger: Trigger fire for each and every record before the triggering DML statement.

After Statement Trigger: Trigger fire only once for each statement after the triggering DML statement executing. After Row Trigger: Trigger fire for each and every record after the triggering DML statement executing.

Uses of Trigger

Triggers supplement the standard capabilities of your database to provide a highly customized database management system. For example, you can use triggers to:

1. Automatically generate derived column values
2. Enforce referential integrity across nodes in a distributed database
3. Enforce complex business rules
4. Provide transparent event logging
5. Provide auditing
6. Maintain synchronous table replicates
7. Gather statistics on table access

References: -

1. Database System Concepts 6th Edition Abraham Silberschatz, Henry F. Korth.